

Sandboxing Framework for Evaluating Microservice Scaling and Bottlenecks

Sathish Adithiyaa SV
IMT2023030
sathishadithiyaa.sv@iiitb.ac.in

Subhash H
IMT2023104
subhash.h@iiitb.ac.in

Abstract—We document a sandboxing and capacity-analysis workflow for the DeathStarBench Hotel Reservation application. Instead of benchmarking the system only end to end, we isolate individual microservices and estimate their Microservice Capacity (MSC), defined as the highest request rate that satisfies a success-rate and latency service-level objective (SLO). Our implementation combines two paths. First, we evaluate **search** inside a dependency sandbox built from captured gRPC traffic, where downstream **geo** and **rate** are replaced by deterministic replay services. Second, we directly stress leaf services such as **geo**, **rate**, **profile**, **recommendation**, **user**, and **reservation** under controlled CPU and replica settings. We also train per-service linear models to predict MSC for unseen configurations. Across collected runs, **search** achieved the highest observed MSC and reached the configured test ceiling of 500 RPS at 500m CPU with two replicas, while **rate** remained the weakest service overall. When we compared sandboxed and live **search** under the same SLO, the sandbox average was 151.1 RPS versus only 19.6 RPS live, showing that sandboxing removed dependency noise and exposed the capacity of **search** itself. A follow-up bottleneck investigation indicates that **rate** is restricted by artificial compute limits at low CPU budgets, and by the overhead of managing concurrent backend requests during cache misses even after those execution limits are removed. Our 150m validation experiment achieved a mean absolute percentage error of 16.4% across six services, showing that the trained model is useful for simple capacity estimation.

Index Terms—microservices, sandboxing, capacity modeling, Kubernetes, performance analysis, DeathStarBench

I. INTRODUCTION

Microservice applications are difficult to capacity-plan because each service has different compute intensity, fan-out behavior, latency sensitivity, and cache behavior. In the Hotel Reservation benchmark, a single user-facing request may traverse several gRPC services and infrastructure components. End-to-end benchmarking is therefore necessary, but it does not directly answer the service-level questions we care about: which microservice is the bottleneck, which resource change helps, and what throughput can we expect from a new deployment configuration.

We address this problem through sandbox-based measurement. We measure a service in controlled isolation while preserving realistic traffic semantics. Following the MSC idea from prior cloud microservice performance-modeling work [1], we define Microservice Capacity as the highest request rate that still satisfies both success-rate and latency SLOs. Our work is not a reproduction of the reference paper.

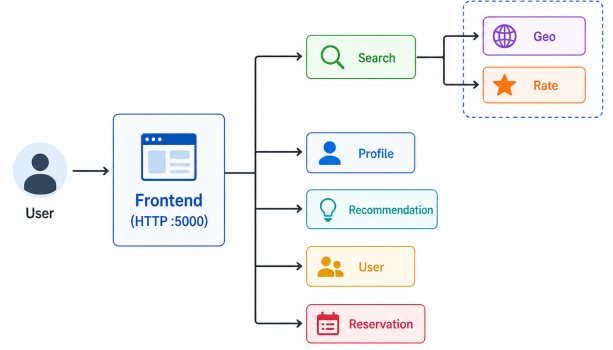


Fig. 1. Simplified Hotel Reservation service interaction architecture used in our experiments. The frontend communicates with multiple services, while **search** uniquely depends on downstream **geo** and **rate**, motivating dependency sandboxing.

Instead, it is an implementation-focused project report describing the logic we built, the experiments we ran, and the insights we obtained on Hotel Reservation from DeathStarBench [?].

Our repository implements a complete workflow that captures gRPC traffic, converts it into deterministic replay fixtures, renders a dedicated Kubernetes sandbox, performs binary-search load testing, trains simple MSC prediction models, and investigates bottlenecks in depth when scaling breaks down. The main contribution of this report is practical: we show how we turned a benchmark application into a reproducible service-level capacity-analysis framework and also analyze how many replicas will be needed for each microservice depending on the target RPS.

II. WORKFLOW AND SANDBOXING DESIGN

A. System Context

Within the portion of Hotel Reservation we studied, **search** depends on **geo** and **rate**. The other evaluated services, namely **geo**, **rate**, **profile**, **recommendation**, **user**, and **reservation**, can be stress-tested directly in our setup. This naturally led us to two complementary workflows:

- 1) dependency sandboxing for **search**, and
- 2) direct leaf-service MSC measurement.

Fig. 1 illustrates the subset of service interactions relevant to our experiments. Among these services, only `search` directly fans out to multiple downstream dependencies, making it the primary candidate for replay-based sandboxing.

This separation is important. If we benchmark `search` against the live cluster, downstream variability from `geo` and `rate` contaminates the result. If we benchmark only leaf services, we miss the behavior of a dependency-heavy microservice. We therefore use both paths and combine their outputs in a single analysis dataset.

It is also the reason we applied sandboxing only to `search`. In our selected set of services, `search` is the only target whose throughput and latency are directly coupled to downstream service behavior. The other services in our study are effectively leaf services in this workflow, so direct MSC measurement is sufficient for them. In other words, sandboxing is not something we applied everywhere by default; we used it precisely where a service-level measurement would otherwise be obscured by dependency effects.

B. Traffic Capture and Replay Fixtures

We added a unary gRPC interceptor to baseline services so that, when capture mode is enabled, each request produces one NDJSON record containing timestamp, service, method, request payload, response payload, gRPC code, and duration. The capture stage is driven by warm-up traffic through the baseline application, so the recorded traces reflect real application calls rather than synthetic protocol-only examples.

We normalize the captured traffic into sandbox fixtures by standardizing JSON payloads, rounding floats, and removing duplicates. By indexing these fixtures by method and normalized request, we achieve deterministic replays that still accurately reflect real-world traffic patterns.

C. Search Sandboxing Procedure

Our `search` experiment runs in six stages:

- 1) send baseline warm-up requests through the frontend,
- 2) capture live `search`, `geo`, and `rate` traffic,
- 3) build `geo` and `rate` replay fixtures plus a `search` request corpus,
- 4) create a fresh sandbox namespace,
- 5) deploy deterministic dummy `geo` and `rate` services, and
- 6) load-test sandboxed `search` under different CPU and replica settings.

The dummy services register through Consul so that `search` continues to resolve downstream dependencies through the normal service-discovery path. This matters because we wanted a replay-based sandbox, not a handwritten shortcut around the application network structure.

Another subtle but important implementation detail is the way requests are distributed across `search` replicas. Once local port-forwards are established, we create one gRPC stub per forwarded pod and cycle through them in round-robin order. We also warm every replica before the measured interval begins. This avoids a biased experiment in which early traffic

sticks to one pod and makes multi-replica configurations look weaker than they actually are. In other words, our replica sweeps exercise the whole `search` pool instead of accidentally measuring just one hot pod.

D. Leaf-Service Experiments

For leaf services, we bypass dependency sandboxing and issue gRPC calls directly to the service under test. Each service has a curated request corpus designed to vary the request shape. For example, `rate` uses different hotel IDs and date windows, `profile` mixes single-hotel and full-batch requests, and `user` checks many seeded Cornell user credentials.

We also intentionally control cache state. For services backed by Memcached, namely `rate`, `profile`, and `reservation`, we flush Memcached before each load step by launching a short-lived helper pod that sends `flush_all` to the cache service. This makes each step start from a colder state and yields a more conservative MSC. The benefit is methodological consistency: capacity numbers are not inflated by warmed cache entries left behind from earlier steps in the binary search.

E. MSC Measurement Logic

We measure MSC using binary search over offered request rate. A candidate rate is accepted only when two conditions hold:

- 1) success rate is at least 0.99, and
- 2) P90 latency is at most 100 ms.

The regression sweeps evaluate CPU values of 100m, 200m, and 500m, and replica counts of 1, 2, and 3. For each configuration, we store the best accepted RPS as MSC together with the observed pod CPU reported by Kubernetes metrics. This produces a dataset that is useful both for visual analysis and for later model fitting.

The request-level timeout remains fixed at 0.3 s for both sandboxed and live `search` runs. This is important for fair comparison. If the timeout or SLO were different across the two paths, then any observed MSC gap could be attributed to measurement settings rather than to the presence or absence of dependency sandboxing.

III. EXPERIMENTAL RESULTS

A. Cross-Service Comparison

Across seven evaluated services, we observed major heterogeneity in MSC. Fig. 2 compares average MSC across all tested configurations. The strongest average services are `user` at 194.44 RPS and `recommendation` at 184.44 RPS. The weakest is `rate` at 86.67 RPS. `search` still ranks strongly at 151.11 RPS even though it is measured in a dependency-aware sandbox rather than as a direct leaf service.

Table I summarizes the best observed configuration for each service. Two points stand out. First, `search` reaches the configured experimental ceiling of 500 RPS at 500m CPU with two replicas, so its true saturation point may be higher. Second, the best configuration is not always the largest one.

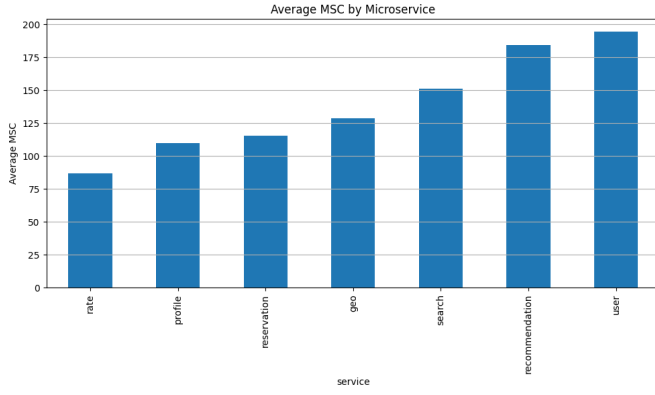


Fig. 2. Average MSC across evaluated microservices.

TABLE I
BEST OBSERVED CONFIGURATION PER SERVICE

Service	CPU (m)	Replicas	Best MSC
Search	500	2	500
Reservation	500	3	380
Geo	500	3	320
Recommendation	200	3	320
Profile	500	2	310
User	500	3	280
Rate	500	3	230

`recommendation`, for example, already reaches 320 RPS at 200m with three replicas, showing that service-level scaling behavior is not simply monotonic in CPU.

B. Search Sandbox Behavior

Fig. 3 shows the main trends for the sandboxed `search` service. At 100m CPU, additional replicas do not help much; the MSC remains low and two replicas give no improvement over one. At 200m CPU, however, the jump from two to three replicas increases MSC from 50 to 145 RPS. This indicates that `search` benefits strongly from additional parallelism once enough aggregate compute is available. At 500m CPU, both the two-replica and three-replica configurations reach the 500 RPS test cap.

These results validate the sandbox methodology itself. We preserved realistic request structure and downstream responses through capture-and-replay, but we removed the run-to-run noise of live downstream services. As a result, the effect of changing CPU or replica count on `search` becomes much easier to interpret.

C. Live Versus Sandboxed Search

To test whether sandboxing genuinely helped us isolate `search`, we also ran the same `search` MSC procedure directly against the live baseline system, where real `geo` and `rate` remained on the critical path. The comparison is decisive. Across the same nine CPU/replica configurations, sandboxed `search` averaged 151.1 RPS, while live `search` averaged only 19.6 RPS. The median MSC advantage of sandboxing was 30 RPS, and the largest gains appeared at the

TABLE II
SANDBOXED VERSUS LIVE SEARCH MSC

CPU (m)	Replicas	Sandbox	Live	Delta
100	1	15	1	+14
100	2	15	10	+5
100	3	25	25	0
200	1	25	15	+10
200	2	50	20	+30
200	3	145	25	+120
500	1	85	25	+60
500	2	500	30	+470
500	3	500	25	+475

higher-resource settings where live `search` remained strongly latency-bound.

Table II makes the pattern clear. At 500m CPU with two replicas, sandboxed `search` reaches the experiment ceiling of 500 RPS, while live `search` reaches only 30 RPS. At 500m with three replicas, the gap is 475 RPS. Even at 200m with three replicas, the sandbox reaches 145 RPS while the live system reaches only 25 RPS.

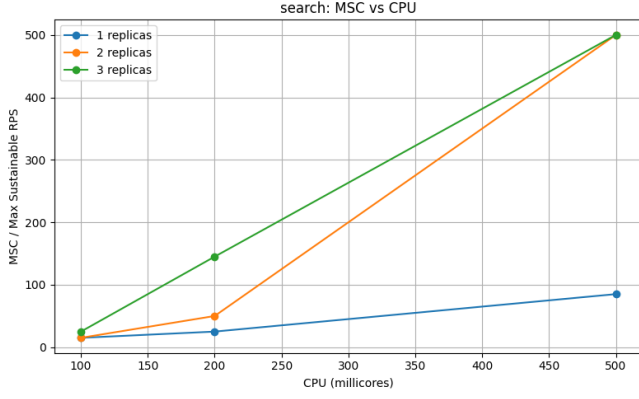
The failure mode also matters. In the live runs, every configuration violated the P90 latency requirement rather than the success-rate requirement. This shows that the live application could often still return requests, but the response-time inflation from downstream dependency behavior prevented us from treating the result as the capacity of `search` itself. In contrast, once `geo` and `rate` were replaced with deterministic replay services, the measured MSC reflected the behavior of `search` much more directly.

This comparison gives us the strongest practical argument for sandboxing in the entire project. Without sandboxing, we would conclude that `search` is a weak service whose capacity remains below 30 RPS in most cases. With sandboxing, we see that `search` can scale to far higher throughput and that much of the live bottleneck actually comes from dependency interaction rather than from the `search` implementation alone. Sandboxing therefore did exactly what we wanted: it allowed us to measure the performance of a single microservice, here `search`, instead of repeatedly measuring the combined behavior of `search`, `geo`, and `rate`.

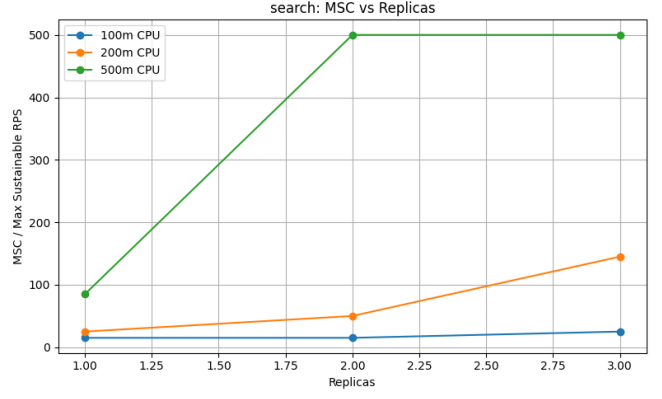
D. Leaf-Service Scaling Trends

Fig. 4 compares two representative leaf-service behaviors. `reservation` scales aggressively at higher CPU budgets and reaches 380 RPS at 500m with three replicas. `rate`, in contrast, scales weakly and shows poor replica efficiency. Even before the detailed bottleneck study, the aggregate plots suggest that `rate` is the most likely system bottleneck.

The contrast between the two services is useful. Both are measured under the same experimental logic, but only `rate` shows persistently weak gains from extra replicas. This tells us that the bottleneck is not an artifact of the measurement harness alone. It is tied to the service’s own runtime behavior.

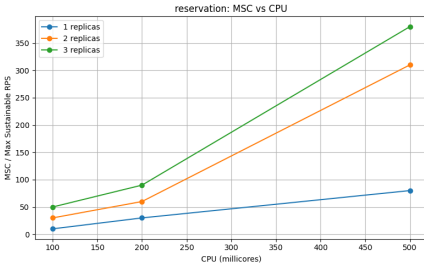


(a) Search MSC versus CPU

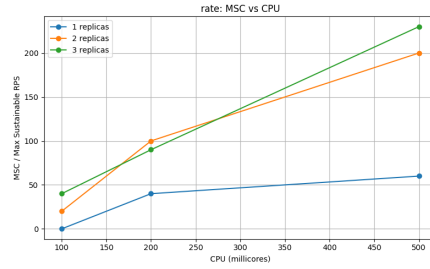


(b) Search MSC versus replicas

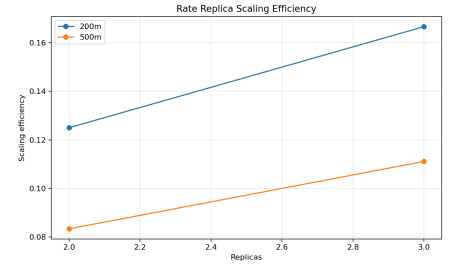
Fig. 3. Capacity trends for sandboxed search.



(a) Reservation MSC versus CPU



(b) Rate MSC versus CPU



(c) Rate replica scaling efficiency

Fig. 4. Representative scaling behavior for reservation and rate.

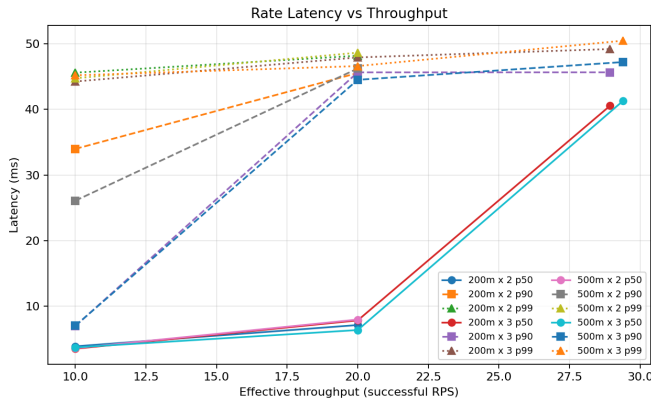


Fig. 5. Latency-throughput behavior for rate.

IV. RATE BOTTLENECK INVESTIGATION

Because *rate* was the weakest service in the aggregate results, we performed a focused bottleneck investigation on four configurations: 200m with two replicas, 200m with three replicas, 500m with two replicas, and 500m with three replicas. These reruns kept the cold-cache policy enabled and collected pod-level CPU statistics, cgroup throttling counters, *perf stat*, and *pidstat* traces.

The focused results sharpen our understanding of the bottleneck in three ways.

First, the 200m configurations show clear CPU throttling. At 200m with two replicas, one pod accumulated 252,873 throttled microseconds and another still saw 16,074. At 200m with three replicas, throttling rose to 422,359 and 49,029 microseconds on two pods. These runs therefore fail under a genuine CPU-control constraint.

Second, the 500m configurations still scale poorly even after throttling disappears. For both 500m with two replicas and 500m with three replicas, the detailed report shows zero throttled microseconds, yet the measured MSC remains only 10 and 20 RPS in the focused cold-cache study. This means the service does not recover simply by removing the CPU cap.

Third, the failure mode at 500m points away from a memory bottleneck and toward a cold-cache coordination problem. The diagnosis report explicitly notes low memory pressure, and the average observed pod CPU remains around 10m even when success-rate violations appear. That combination is important: the service is not failing because it fully consumes CPU or memory. It is failing while spending substantial time waiting on backend work, request fan-out, cache-miss handling, serialization, or network coordination. This interpretation is also consistent with the measured run-queue wait times being very small while the service still fails to scale linearly.

TABLE III
VALIDATION RESULTS AT 150M CPU

Service	Replicas	Actual MSC	Predicted MSC
Geo	2	70	74.7
Rate	2	40	42.6
Profile	2	40	53.9
Recommendation	2	170	147.9
User	2	200	160.2
Reservation	2	40	47.1

Taken together, these observations make our explanation more precise than simply saying that `rate` is “slow.” At low CPU budgets, Linux cgroup throttling is a direct limiter. At higher CPU budgets, the bottleneck shifts to the cold-cache execution path and the service’s coordination overhead per request. In practical terms, adding replicas alone is not enough; improving cache-hit behavior, reducing downstream work per request, or restructuring request processing is more likely to help.

This also helps explain the live-versus-sandbox `search` gap. Because live `search` must wait on real `rate` responses, the weak scaling and latency sensitivity of `rate` are inherited by the live `search` path. Once we replace `rate` with deterministic replay, that inherited bottleneck is removed, and the measured throughput moves much closer to the intrinsic behavior of `search`.

V. PREDICTIVE MSC MODELING

A. Per-Service Model

After collecting the regression data, we added a lightweight prediction layer to estimate MSC for unseen configurations. Instead of fitting one global model for all services, we train one model per service. This reflects our earlier observation that scaling behavior differs substantially across services.

The current model uses only features available at deployment time:

- 1) CPU millicores per pod, and
- 2) replica count.

For each service, we fit either a constant model when only one clean point is available or a linear model of the form

$$MSC = \beta_0 + \beta_1 \cdot CPU + \beta_2 \cdot Replicas.$$

B. Validation on Unseen 150m CPU Runs

To test whether the learned models generalize beyond the training grid, we ran a separate validation experiment at 150m CPU, a value not present in the original 100m/200m/500m sweep. Table III summarizes the results for the six leaf services included in the validation dataset. Across these held-out points, the mean absolute percentage error (MAPE) is 16.4%.

The model performs best on `geo` and `rate`, where the prediction errors are 6.8% and 6.5%, respectively. It performs worst on `profile`, where the error rises to 34.8%. This pattern is reasonable: a two-feature linear model can approximate smooth scaling trends, but it cannot capture stronger non-linearities, cache effects, or saturation plateaus. Even so, a

16.4% mean error is good enough to support coarse capacity planning and to narrow the configurations worth testing next.

There is another useful interpretation of this validation result. We are not claiming that a two-feature linear model can replace direct measurement. Instead, the model acts as a first-pass estimator. It can suggest whether a new CPU/replica point is likely to be near a useful operating region before we spend time running a full experiment. In that sense, modeling complements sandboxing and stress testing rather than replacing either one.

VI. DISCUSSION

Three main conclusions emerged from the repository and our experiments.

First, sandboxing is most valuable for dependency-heavy services. The `search` pipeline shows that capture-and-replay can preserve realistic semantics while making experiments more reproducible and much easier to reason about than full end-to-end reruns.

Second, MSC is highly service-specific. Within the same application and under the same SLO, some services scale well with more resources, while others do not. This validates the need for per-service capacity planning rather than a single autoscaling rule for the entire application.

Third, raw resource provisioning is not the same as real scaling. Our results repeatedly show that adding CPU or replicas can help only when the service architecture is ready to use them. Where the runtime is dominated by cache misses, synchronization, or request amplification, additional resources have diminishing returns.

A fourth conclusion follows from the live-versus-sandbox comparison. End-to-end measurement alone can be misleading when the goal is per-service capacity analysis. Live `search` looked severely constrained across almost all configurations, but the sandboxed results showed that `search` itself was not the main limiter. In other words, the experiment design determines what question the result answers. The live runs answer “how does the whole dependency chain behave when `search` is resized?” The sandboxed runs answer “what is the capacity of `search` itself under realistic request semantics?” For this project, we needed both questions, but only sandboxing let us answer the second one correctly.

From a systems perspective, this separation is useful for tuning decisions. If the live result is poor but the sandboxed result is strong, then optimization effort should focus on the dependencies or their interaction patterns. If both results are poor, then the service under test is likely a primary bottleneck. Our data places `search` in the first category and `rate` in the second.

Our current implementation also has limitations. The experiments are Minikube-first, so the exact numeric results are cluster-specific and SLO specific. Through are experiments we found that the MSC is heavily dependent on SLO and this highlights how crucial an accurate SLO definition is to achieve realistic analysis. Finally, the best `search` run is

right-censored at 500 RPS because that was the configured maximum test rate.

VII. FUTURE WORK

Future work can extend the system in two directions. One direction is depth: add richer prediction models that incorporate memory, observed CPU, or non-linear interaction terms. A second direction is automation: use the measured MSC data to recommend deployment configurations or to guide autoscaling policies more directly. Even in its current form, however, the workflow already demonstrates that service-level sandboxing and lightweight predictive modeling can work together effectively.

VIII. CONCLUSION

We developed a practical sandbox-based capacity-analysis framework for the DeathStarBench Hotel Reservation application. The framework captures real gRPC traffic, normalizes it into replay fixtures, deploys a Kubernetes sandbox for search, measures MSC through binary-search load testing, and complements the main sweeps with both predictive modeling and targeted bottleneck diagnosis.

Our results show four clear outcomes. `search` scales strongly when its downstream dependencies are sandboxed through deterministic replay. Live `search`, by contrast, remains strongly P90-latency-bound, which confirms that sandboxing is necessary if we want to measure the capacity of `search` itself rather than the capacity of the whole dependency chain. `reservation` scales well with added CPU and replicas. `rate` is the dominant bottleneck candidate because of throttling at low CPU limits and cold-cache coordination overhead at higher limits. The added prediction model further shows that simple per-service linear fitting can already provide useful MSC estimates for unseen CPU settings, achieving 16.4% MAPE on the 150m validation runs.

Overall, we found that sandbox-based microservice analysis provides much deeper operational insight than conventional end-to-end benchmarking alone. It lets us separate dependency effects from service effects, reason about replica scaling more accurately, and move from raw measurements toward practical capacity prediction.

REFERENCES

- [1] A. Jindal, V. Podolskiy, and M. Gerndt, "Performance modeling for cloud microservice applications," in *Proc. ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2019.
- [2] Github repository: <https://github.com/SathishAdithiyaaSV/SSP-Project>